Solution: Reverse Words in a String II

Problem Statement

Given a character array `s`, return the updated array after reversing the order of `words` in the array.

A word is defined as a sequence of `non-space characters`, and the words are separated by `one space` character. The input array does not contain any leading or trailing spaces, and the words are always separated by a `single space`.

Note: Your code must solve the problem without using any extra space.

Examples

- **Example 1**:
  - **Input**: `["h","e","l","l","o"," ","w","o","r","l","d"]`
  - **Expected Output**: `["w","o","r","l","d"," ","h","e","l","l","o"]`
  - **Justification**: The original string is "hello world". Reversing the words gives "world hello".
- **Example 2**:
  - **Input**: `["m","o","r","n","i","n","g"," ","s","k","y"," ","i","s"," ","b","r","i","g","h","t"]`
  - **Expected Output**: `["b","r","i","g","h","t"," ","i","s"," ","s","k","y"," ","m","o","r","n","i","n","g"]`
  - **Justification**: The original string is "morning sky is bright". Reversing the words gives "bright is sky morning".
- **Example 3**:
  - **Input**: `["a","l","o","n","e"]`
  - **Expected Output**: `["a","l","o","n","e"]`
  - **Justification**: There's only one word "alone", so the reverse is the same as the original.

Solution

To solve this problem, the approach involves two main steps: first, reverse the entire array to get the words in the reverse order but with the characters of each word also reversed. Second, reverse the characters of each word to correct their orientation. This method works because reversing the entire array places the words in their final positions relative to each other, and then reversing the characters within each word ensures that the words are oriented correctly. This approach is effective because it allows us to achieve the desired result with a simple, two-pass solution that operates directly on the array in-place, ensuring that no additional space is required beyond minimal temporary variables.

This method is chosen for its efficiency and simplicity. By manipulating the array in place, we avoid the overhead of extra memory allocation that would be required for a solution that constructs a new array or uses additional data structures. This makes the algorithm particularly suitable for large inputs and constraints where space efficiency is a concern.

Step-by-Step Algorithm

1. **Reverse the entire array of characters.**
   - Identify the start and end indexes of the array.
   - Swap characters at the start and end indexes.
   - Increment the start index and decrement the end index.
   - Continue this process until the start index is greater than or equal to the end index.
2. **Identify the bounds of each word.**
   - Initialize a variable to track the start of the current word.
   - Iterate through the array with an index variable.
     - If a space character or the end of the array is encountered, the current index marks the end of a word.
3. **Reverse each word identified in step 2.**
   - For each word, repeat the reversal process used in step 1, but apply it only to the characters within the bounds of the word.
     - Swap characters at the start and end of the word.
     - Increment the word's start index and decrement the word's end index.
     - Continue this process until the start index of the word is greater than or equal to the end index of the word.

Algorithm Walkthrough

Consider the input:

```
["m","o","r","n","i","n","g"," ","s","k","y"," ","i","s","
","b","r","i","g","h","t"]
```

1. **Reverse the entire array**:
   - Initial array:
     `["m", "o", "r", "n", "i", "n", "g", " ", "s", "k", "y", " ", "i", "s", " ", "b", "r", "i", "g", "h", "t"]`
   - After reversing:
     `["t", "h", "g", "i", "r", "b", " ", "s", "i", " ", "y", "k", "s", " ", "g", "n", "i", "n", "r", "o", "m"]`
2. **Identify bounds of each word and reverse**:
   - Word 1 ("t", "h", "g", "i", "r", "b"): Reverse to get `["b", "r", "i", "g", "h", "t"]`.
   - Word 2 ("s", "i"): Reverse to get `["i", "s"]`.
   - Word 3 ("y", "k", "s"): Reverse to get `["s", "k", "y"]`.
   - Word 4 ("g", "n", "i", "n", "r", "o", "m"): Reverse to get `["m", "o", "r", "n", "i", "n", "g"]`.
3. **Final array**:
   - `["b", "r", "i", "g", "h", "t", " ", "i", "s", " ", "s", "k", "y", " ", "m", "o", "r", "n", "i", "n", "g"]`

Code

Java

```java
public class Solution {
    // Method to reverse words in an array
    public char[] reverseWords(char[] s) {
        // Step 1: Reverse the entire array
        reverse(s, 0, s.length - 1);

        // Step 2 & 3: Find each word's bounds and reverse each word
        int start = 0; // Start index of a word
        for (int i = 0; i <= s.length; i++) {
            if (i == s.length || s[i] == ' ') {
                reverse(s, start, i - 1); // Reverse the current word
                start = i + 1; // Update the start index for the next word
            }
        }
        return s;
    }

    // Helper method to reverse a portion of the array
    private void reverse(char[] s, int start, int end) {
        while (start < end) {
            char temp = s[start];
```

Run Code Result Stdout

You must run your code first

Execute

Complexity Analysis

- **Time Complexity**: The overall time complexity of the algorithm is $O(n)$, where n is the length of the input array. This is because each character is visited twice at most - once during the entire array reversal and once during the reversal of each word.
- **Space Complexity**: The space complexity is $O(1)$, as the reversal is done in place, and no additional space proportional to the input size is used.

← Previous
Reverse Words In A String II

Next →
Longest Arithmetic Subsequence (Medium)

☐ Mark as Completed